ECG Beat Classification with Adaptive Re-learning

System Design Document

Implementation overview and design choices

1. Objective and Scope

The project builds a five-class ECG beat classifier and a small adaptive re-learning workflow. It uses the MIT-BIH Arrhythmia Database, extracts 270-sample beats around annotated locations, trains a PyTorch 1D CNN, stores predictions in SQLite, applies simulated reviewer corrections, and evaluates a re-learned model before making it active.

The implementation was developed in Google Colab and uses Google Drive for the dataset and saved artifacts.

2. Problem Definition

The first part of the problem is to classify ECG beats into the five AAMI-style groups N, S, V, F, and Q. The second part is to update the classifier when reviewer corrections are received without using the final evaluation data for training.

The main engineering issues are class imbalance, record-level leakage, a small correction set, possible catastrophic forgetting, and the need for a controlled model promotion step.

3. Dataset and Data Preparation

3.1 Dataset

The project uses the MIT-BIH Arrhythmia Database and its beat annotations. WFDB is used to read the records and annotations.

Class	Meaning
N	Normal
S	Supraventricular
V	Ventricular
F	Fusion
Q	Unknown / other

3.2 ECG preprocessing

A third-order Butterworth bandpass filter is applied before beat extraction.

```
High-pass: 0.5 Hz    |    Low-pass: 40 Hz
```

Each beat is represented using 90 samples before the annotation and 180 samples after it, giving 270 samples per beat.

3.3 Train / validation / test separation

Records are separated into DS1 and DS2 at the record level. Within DS1, 17 records are used for training and 5 for validation. The notebook checks that records do not overlap between the resulting sets.

DS2 is kept as the final locked evaluation set so that the before/after model comparison uses the same data.

3.4 Class imbalance

The training data is strongly imbalanced. S and F are oversampled in DS1 training. Validation is not oversampled and DS2 is not modified. Moderated square-root inverse-frequency class weights are also used in the training loss.

4. Baseline Model

A compact PyTorch 1D CNN is used because the input is a fixed-length one-dimensional ECG waveform. It learns local waveform patterns while remaining small enough for the available Colab environment.

```
270-sample beat -> 1D CNN -> 5 class outputs (N, S, V, F, Q)
```

The model uses convolutional layers with normalization, activation, pooling and dropout, followed by global average pooling and fully connected layers. Training uses Adam with weighted cross-entropy. The baseline model is cnn_v1.0.

5. Part B - Inference and Persistence

5.1 Baseline inference

The baseline model runs inference on the DS2 test loader and stores the resulting predictions in SQLite.

5.2 Stored prediction fields

Field	Purpose
unique_beat_id	Unique beat identifier
source_record	Original ECG record
timestamp	Stored event time
input_signal_ref	Input reference
predicted_label	Model prediction
confidence	Maximum softmax probability
model_version	Model that produced the prediction

The final B3 implementation keeps the actual source-record identifier for each DS2 beat.

5.3 External inference

C8 reads the active model from the model registry, applies the inference preprocessing to an external 270-sample beat, and returns the predicted class and confidence.

6. Part C - Adaptive Re-learning

6.1 C0 - Classified beat inspection

C0 inspects existing classification records used by the review and audit workflow.

6.2 C1 - Audit trail

C1 records beat ID, old label, new label, reviewer information, correction time, and original model version. A duplicate correction for the same adaptation ID is skipped.

6.3 C2 - Reviewer correction

Reviewer corrections are simulated in the notebook using reference annotations for incorrectly classified DS1 adaptation beats. The corrected labels are written to the audit trail and used by the adaptation step.

6.4 C3 - Re-learning trigger

The system does not retrain after every correction. The trigger uses correction count, minority-class correction activity, and correction drift. The demonstration is configured with a threshold of 5 unique corrections, 2 minority-class corrections, and a 1% drift threshold.

6.5 C4 - Locked DS2 and adaptation data

C4 keeps the original DS2 evaluation data separate from the DS1 adaptation data. The locked DS2 set contains 49,692 beats and is not used to build the re-learning dataset.

6.6 C5 - Adaptive re-learning

C5 starts from cnn_v1.0 and trains with reviewer-corrected adaptation examples plus class-aware DS1 replay. The output model is cnn_v2.0. Replay is used to retain exposure to the original DS1 training distribution.

6.7 C6 - Before / after evaluation

C6 evaluates cnn_v1.0 and cnn_v2.0 on the same locked DS2 set. Accuracy, Macro F1, per-class metrics, the classification report, and the confusion matrix are reported.

6.8 C6.5 - Forgetting check

C6.5 compares V1 and V2 using per-class F1 and recall changes. F and Q are still reported; their unchanged zero F1 is treated as a performance limitation.

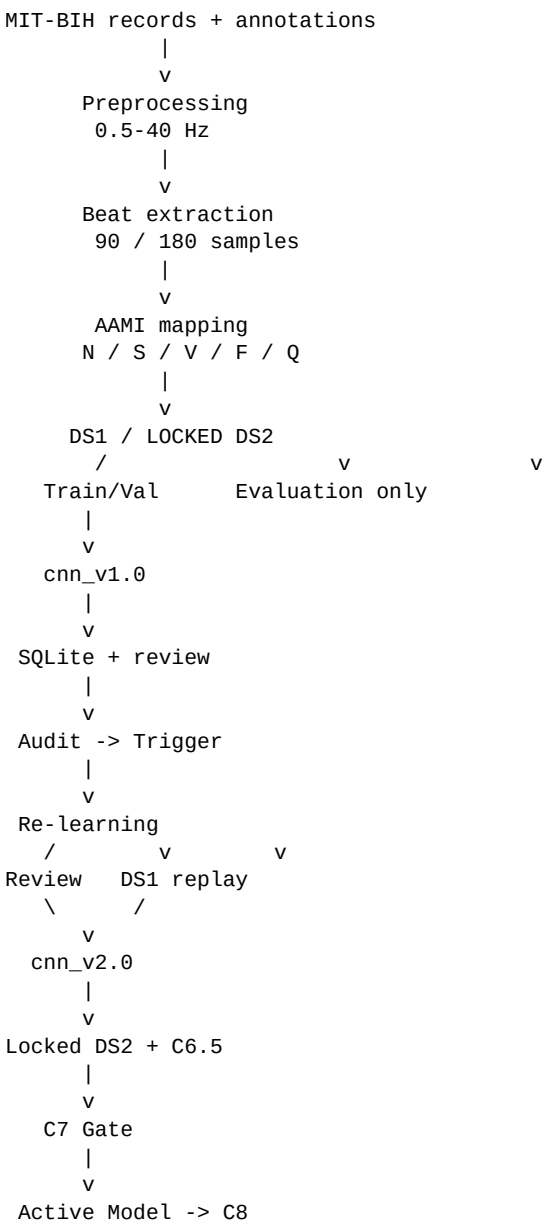
6.9 C7 - Promotion gate

C7 checks Accuracy, Macro F1 and per-class F1 before promotion. The maximum allowed regression is 1 percentage point. When the gate passes, cnn_v2.0 is marked active in the model registry.

6.10 C8 - Active model inference

C8 gets the active model from the registry and uses it for external ECG beat inference.

7. System Architecture



The main separation in the design is between the adaptation path and the locked DS2 evaluation path.

8. Results

Metric	cnn_v1.0	cnn_v2.0	Change
Accuracy	81.0794%	83.8545%	+2.7751 pp
Macro F1	0.3031	0.3145	+0.0114

Class	V1 F1	V2 F1	Change
N	0.8940	0.9106	+0.0166
S	0.0441	0.0463	+0.0022
V	0.5774	0.6154	+0.0380
F	0.0000	0.0000	+0.0000
Q	0.0000	0.0000	+0.0000

The final C7 run passed the Accuracy, Macro F1 and per-class gates and promoted cnn_v2.0. C6.5 found no evidence of catastrophic forgetting within the configured tolerance.

C8 loaded cnn_v2.0 from the registry and predicted N for an external 270-sample ECG beat with confidence 0.9943.

9. Challenges and Limitations

- Class imbalance remains a major issue; F and Q have zero F1 on the locked DS2 evaluation set.
- The adaptation set is small, so the re-learning result should be viewed as a workflow demonstration rather than large-scale continual learning.
- The trigger thresholds are fixed engineering values and were not learned from long-term reviewer data.
- Reviewer corrections are simulated in the notebook; there is no separate reviewer interface.
- The current implementation is notebook-based rather than a packaged production service.
- SQLite is suitable for this small workflow, but the database-locking issue seen during development shows that stronger concurrency handling would be needed at larger scale.

10. Future Improvements

- Split the notebook into separate preprocessing, training, inference, persistence, and re-learning modules.
- Add a lightweight reviewer interface for submitting corrections.
- Improve F and Q using better data coverage and minority-class methods.
- Calibrate the trigger thresholds using real reviewer correction history.
- Add automated tests for data separation, audit records, model versioning, and the promotion gate.
- Use stronger database transaction/concurrency handling for larger workloads.
- Evaluate several consecutive correction and re-learning cycles.

11. Reproducibility

Random seed: 42. The DS1/DS2 record split is fixed, and the V1/V2 comparison uses the same locked DS2 dataset. Model versions are cnn_v1.0 and cnn_v2.0. C8 reads the active version from the model registry. The MIT-BIH dataset is kept outside the repository and loaded from the configured Google Drive location.

12. Conclusion

The project connects ECG preprocessing, classification, persistence, reviewer correction, adaptive re-learning, locked evaluation, forgetting analysis, and model promotion in one notebook workflow. The re-learned model performs better on the locked DS2 set on the reported overall metrics, while the weak F and Q results and the notebook-based implementation remain important limitations.